4. Presenter y Main

Las dos piezas que **conectan** todo lo demás.

Presenter: el intermediario

`Presenter.java`

El modelo y la vista no se conocen. El Presenter es el único que conoce a los dos.

```
public class Presenter implements IPresenter {
    private final IModel model;
    private final IView view;
```

Fíjate en que guarda `IModel` e `IView` (los contratos), no `Model` ni `View`. Solo usa los métodos que los contratos prometen.

El constructor

`Presenter.java:15`

Recibe el modelo y la vista ya creados y los guarda. No los crea él: los crea el `Main`.

`start()`: poner todo en marcha

`Presenter.java:21`

```
view.setPresenter(this);
model.start();
view.start();
```

El orden es importante:

1. **`view.setPresenter(this)`** — la vista necesita conocer al Presenter *antes* de arrancar, porque en su `start()` ya le pide una imagen. Si se hiciera al revés, `presenter` sería `null` y el programa fallaría.
2. **`model.start()`** — arranca el hilo del fuego, para que ya haya algo que enseñar.
3. **`view.start()`** — abre la ventana y empieza a refrescar.

`getImage()`: pasar la petición

`Presenter.java:28`

```
return model.getImage();
```

La vista le pide la imagen al Presenter, y el Presenter se la pide al modelo. Es un simple "paso de mano": el Presenter no calcula nada.

Los tres contratos

Las interfaces dicen **qué** hay que hacer, no **cómo**:

Interfaz	Métodos	La implementa
`IModel`	`update()`, `start()`, `getImage()`	`Model`
`IView`	`setPresenter(...)`, `start()`	`View`
`IPresenter`	`start()`, `getImage()`	`Presenter`

Fíjate en una cosa: `IPresenter` tiene `getImage()` porque **la vista lo necesita**. La vista solo conoce el contrato `IPresenter`, así que todo lo que quiera pedir tiene que estar ahí.

Main: el punto de entrada

`Main.java`

```
SwingUtilities.invokeLater(() -> {
    IModel model = new Model();
    IView view = new View();
    IPresenter presenter = new Presenter(model, view);

    presenter.start();
});
```

Hace solo dos cosas:

1. **Crea las tres piezas** y las une (el Presenter recibe el modelo y la vista).
2. **Llama a** `presenter.start()`, que lo pone todo en marcha.

Por qué `invokeLater`

Las ventanas de Swing tienen que crearse en un hilo especial propio de Swing (el "hilo de eventos"). `SwingUtilities.invokeLater` ejecuta el bloque en ese hilo. Es la forma segura de crear una ventana.

Por qué las variables son del tipo de la interfaz

```
IModel model = new Model();
```

Se declara como `IModel` (el contrato), aunque el objeto sea un `Model`. Así el resto del código solo depende del contrato. Aquí en el `Main` es el único sitio donde se nombran las clases concretas (`Model`, `View`, `Presenter`).

Resumen en una frase por clase

Clase	Resumen
`Main`	Crea las tres piezas y arranca
`Presenter`	Conecta modelo y vista, y pasa las peticiones
`Model`	Hace avanzar el fuego en un hilo
`Fire`	Guarda las temperaturas y las hace subir
`ColorTargetDTO`	Un color clave de la paleta
`Palette`	Convierte temperatura en color
`Render`	Convierte las temperaturas en una imagen
`View`	La ventana y el hilo que la refresca
`Viewer`	El lienzo que dibuja la imagen